```
!pip install -q sentence-transformers scikit-learn pandas
```

```
import pandas as pd
import numpy as np

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
faq_data = {
    "question": [
        "What is the college fee structure?",
        "How can I apply for admission?",
        "What are the library timings?",
        "How can I get my examination timetable?",
        "What is the minimum attendance requirement?",
        "Where is the placement cell located?",
        "How can I get a bonafide certificate?",
        "Does the college provide hostel facilities?",
        "What are the working hours of the college?",
        "How can I contact the examination department?",
        "How do I pay my tuition fees?",
        "When will the semester exams start?",
        "How can I apply for a scholarship?",
        "Where can I get my ID card?",
        "How can I contact my faculty?"
    ],

    "answer": [
        "Check the fee structure on the college website or contact the accou
        "Apply through the official college admission portal.",
        "The library is open from 9 AM to 5 PM.",
        "The examination timetable is available on the college website.",
        "Students must maintain the minimum attendance required by universit
        "The placement cell is located in the college administration buildin
        "Request a bonafide certificate from the college office.",
        "Contact the student section for hostel information.",
        "College working hours are generally from 9 AM to 5 PM.",
        "Contact the examination department through the college office.",
        "Tuition fees can be paid through the college fee payment portal.",
```

```
        "Semester examination dates are published in the examination timetab
        "Contact the scholarship department for application details.",
        "Collect your ID card from the college administration office.",
        "You can contact faculty through the department office."
    ]
}

df = pd.DataFrame(faq_data)

print("College FAQ Dataset Created!")
print("Total FAQs:", len(df))

df.head(15)
```

```
College FAQ Dataset Created!
Total FAQs: 15
```

	question	answer
0	What is the college fee structure?	Check the fee structure on the college website...
1	How can I apply for admission?	Apply through the official college admission p...
2	What are the library timings?	The library is open from 9 AM to 5 PM.
3	How can I get my examination timetable?	The examination timetable is available on the ...
4	What is the minimum attendance requirement?	Students must maintain the minimum attendance ...
5	Where is the placement cell located?	The placement cell is located in the college a...
6	How can I get a bonafide certificate?	Request a bonafide certificate from the colleg...
7	Does the college provide hostel facilities?	Contact the student section for hostel informa...
8	What are the working hours of the college?	College working hours are generally from 9 AM ...
9	How can I contact the examination department?	Contact the examination department through the...
10	How do I pay my tuition fees?	Tuition fees can be paid through the college f...
11	When will the semester exams start?	Semester examination dates are published in th...
12	How can I apply for a scholarship?	Contact the scholarship department for applica...
13	Where can I get my ID card?	Collect your ID card from the college administ...
14	How can I contact my faculty?	You can contact faculty through the department...

Next steps: Generate code with df | New interactive sheet

```
model = SentenceTransformer("all-MiniLM-L6-v2")

print("Sentence Transformer model loaded successfully!")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req
modules.json: 100%                                                  349/349 [00:00<00:00, 11.7kB/s]
config_sentence_transformers.json: 100%                             116/116 [00:00<00:00, 1.73kB/s]
README.md: 100%                                                     10.5k/10.5k [00:00<00:00, 142kB/s]
sentence_bert_config.json: 100%                                     53.0/53.0 [00:00<00:00, 1.30kB/s]
config.json: 100%                                                   612/612 [00:00<00:00, 15.3kB/s]
model.safetensors: reconstructing file: 100%                        90.9MB / 90.9MB, 8.70MB/s
model.safetensors: downloading bytes:                               85.0MB, 7.92MB/s
Loading weights: 100%                                               103/103 [00:00<00:00, 1401.57it/s]
tokenizer_config.json: 100%                                         350/350 [00:00<00:00, 5.09kB/s]
vocab.txt: 100%                                                     232k/232k [00:00<00:00, 2.44MB/s]
tokenizer.json: 100%                                                466k/466k [00:00<00:00, 7.66MB/s]
special_tokens_map.json: 100%                                       112/112 [00:00<00:00, 2.26kB/s]
config.json: 100%                                                   190/190 [00:00<00:00, 4.19kB/s]
Sentence Transformer model loaded successfully!
```

```
faq_questions = df["question"].tolist()

faq_embeddings = model.encode(faq_questions)

print("FAQ Embeddings Generated Successfully!")

print("Embedding Shape:", faq_embeddings.shape)
```

```
FAQ Embeddings Generated Successfully!
Embedding Shape: (15, 384)
```

```
def find_similar_question(user_question):

    # Convert user question into embedding
    query_embedding = model.encode([user_question])

    # Calculate cosine similarity
    similarity_scores = cosine_similarity(
        query_embedding,
        faq_embeddings
```

```
    )[0]

    # Find the most similar question
    best_index = np.argmax(similarity_scores)

    best_score = similarity_scores[best_index]

    print("\nUser Question:", user_question)

    print("\nMost Similar FAQ:",
          df.iloc[best_index]["question"])

    print("\nAnswer:",
          df.iloc[best_index]["answer"])

    print("\nSimilarity Score:",
          round(float(best_score), 4))

    return best_index, best_score
```

```
user_question = "How much does studying in this college cost?"

find_similar_question(user_question)
```

```
User Question: How much does studying in this college cost?

Most Similar FAQ: What is the college fee structure?

Answer: Check the fee structure on the college website or contact the accounts d

Similarity Score: 0.7216
(np.int64(0), np.float32(0.72159415))
```

```
find_similar_question(
    "When can students use the library?"
)

find_similar_question(
    "How do I get admission in this college?"
)

find_similar_question(
    "Where can I find the exam dates?"
)
```

```
User Question: When can students use the library?

Most Similar FAQ: What are the library timings?

Answer: The library is open from 9 AM to 5 PM.

Similarity Score: 0.5136

User Question: How do I get admission in this college?

Most Similar FAQ: How can I apply for admission?

Answer: Apply through the official college admission portal.

Similarity Score: 0.7972

User Question: Where can I find the exam dates?

Most Similar FAQ: How can I get my examination timetable?

Answer: The examination timetable is available on the college website.

Similarity Score: 0.7327
(np.int64(3), np.float32(0.7327192))
```

```
def find_top_3_questions(user_question):

    query_embedding = model.encode([user_question])

    scores = cosine_similarity(
        query_embedding,
        faq_embeddings
    )[0]

    top_indices = np.argsort(scores)[::-1][:3]

    print("User Question:", user_question)
    print("\nTop 3 Matching Questions:\n")

    for rank, index in enumerate(top_indices, start=1):

        print("Rank:", rank)

        print("Question:", df.iloc[index]["question"])

        print("Answer:", df.iloc[index]["answer"])

        print("Similarity:",
              round(float(scores[index]), 4))
```

```
        print("-" * 50)


find_top_3_questions(
    "How much money do I need to pay for college?"
)
```

```
User Question: How much money do I need to pay for college?

Top 3 Matching Questions:

Rank: 1
Question: What is the college fee structure?
Answer: Check the fee structure on the college website or contact the accounts d
Similarity: 0.6496
--------------------------------------------------
Rank: 2
Question: How do I pay my tuition fees?
Answer: Tuition fees can be paid through the college fee payment portal.
Similarity: 0.5604
--------------------------------------------------
Rank: 3
Question: What are the working hours of the college?
Answer: College working hours are generally from 9 AM to 5 PM.
Similarity: 0.4444
--------------------------------------------------
```

```
!pip install -q gradio
```

```
import gradio as gr

def faq_search(question):

    if not question.strip():
        return "Please enter a question."

    query_embedding = model.encode([question])

    scores = cosine_similarity(
        query_embedding,
        faq_embeddings
    )[0]

    best_index = np.argmax(scores)
    best_score = scores[best_index]

    if best_score < 0.35:
        return "No sufficiently similar FAQ found. Please contact the college
```

```
        return "No sufficiently similar FAQ found. Please contact the colleg

    return (
        f"Matching Question:\n{df.iloc[best_index]['question']}\n\n"
        f"Answer:\n{df.iloc[best_index]['answer']}\n\n"
        f"Similarity Score: {best_score:.4f}"
    )


demo = gr.Interface(
    fn=faq_search,
    inputs=gr.Textbox(
        label="Enter your college question",
        placeholder="Example: How much are college fees?"
    ),
    outputs=gr.Textbox(label="FAQ Search Result"),
    title="College FAQ Semantic Similarity Application",
    description="Find college answers using AI semantic search."
)

demo.launch(share=True)
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in lau
* Running on public URL: https://7ab72d2d0acaad2a61.gradio.live

This share link is temporary and will last for up to 1 week (best effort). For f
```

